

4.1 Machine Learning

徐英泽 2024011267

[返回引导页](#)

1. 我们提供了一个qPCR数据集qPCR_data.csv，第1列为sample id，第2-12列为特征(11个基因的表达量)，第13列为样本标签(负例为健康人:NC,正例为肝癌病人:HCC)。

数据预处理:

```
# 载入必要的R包
library(glmnet)
library(caret)
library(pROC)
library(mlbench)

# 导入数据
raw_data <- read.csv("C:/Users/54727/Desktop/bioinformatic back up/bioinformatic/4.1 Machine Learning/qPCR_data.csv", header = TRUE)
head(raw_data)
```

	sample_id	SNORD3B	HULC	LINC01226	hsa_circ_0073052	hsa_circ_0080695	miR.21
	miR.26a	miR.27a	miR.122				
1	HCC_ShH-332406	3.072431	-1.554841	1.1312731	-0.06643703	-6.124729	8.1331854
	-0.6244188	0.2376365	9.4565002				
2	HCC_ShH-333568	-1.559677	-2.728607	-0.9597084	0.66675478	-7.979978	0.8054626
	0.1381638	1.1987614	3.4747719				
3	HCC_ShH-321590	-2.077954	-8.188078	-6.7839831	NA	NA	-1.1054335
	-2.0461284	-1.0872934	0.5445430				
4	HCC_ShH-335701	7.648093	2.122047	5.2242226	1.93861532	-4.897092	-1.3748004
	-1.3089088	-2.6353329	1.4379535				
5	HCC_ShH-335789	2.819371	-4.583385	-3.3328530	-0.51496394	-8.407561	-1.6972703
	-1.0836709	-1.0498402	0.8840812				
6	HCC_ShH-214851	2.943885	-3.812490	0.2933004	-0.22909032	-9.753296	-0.5045257
	-1.1497174	-0.8898168	-2.8814714				
	miR.192	miR.223	label				
1	-0.1583774	-5.195042	HCC				
2	-1.3592913	-6.533736	HCC				
3	-3.7506084	-9.531275	HCC				
4	-5.0366864	-8.267998	HCC				
5	-4.3496537	-8.383437	HCC				
6	-4.3981321	-7.467220	HCC				

```
# 分别计算HCC与NC的平均值
y <- raw_data$label
x <- raw_data[,2:12]
hcc_idx <- which(y == "HCC")
nc_idx <- which(y == "NC")
means_hcc <- colMeans(x[hcc_idx, ], na.rm = TRUE)
```

```

means_nc <- colMeans(x[nc_idx, ], na.rm = TRUE)

# 使用for循环与matrix函数用均值填补缺失值
x[hcc_idx, ][is.na(x[hcc_idx, ])] <-
+   matrix(rep(means_hcc, each = length(hcc_idx)),
+         nrow = length(hcc_idx))[is.na(x[hcc_idx, ])]

x[nc_idx, ][is.na(x[nc_idx, ])] <-
+   matrix(rep(means_nc, each = length(nc_idx)),
+         nrow = length(nc_idx))[is.na(x[nc_idx, ])]

# 数据归一化
x <- scale(x, center = TRUE, scale = TRUE)

```

使用PCA进行数据可视化:

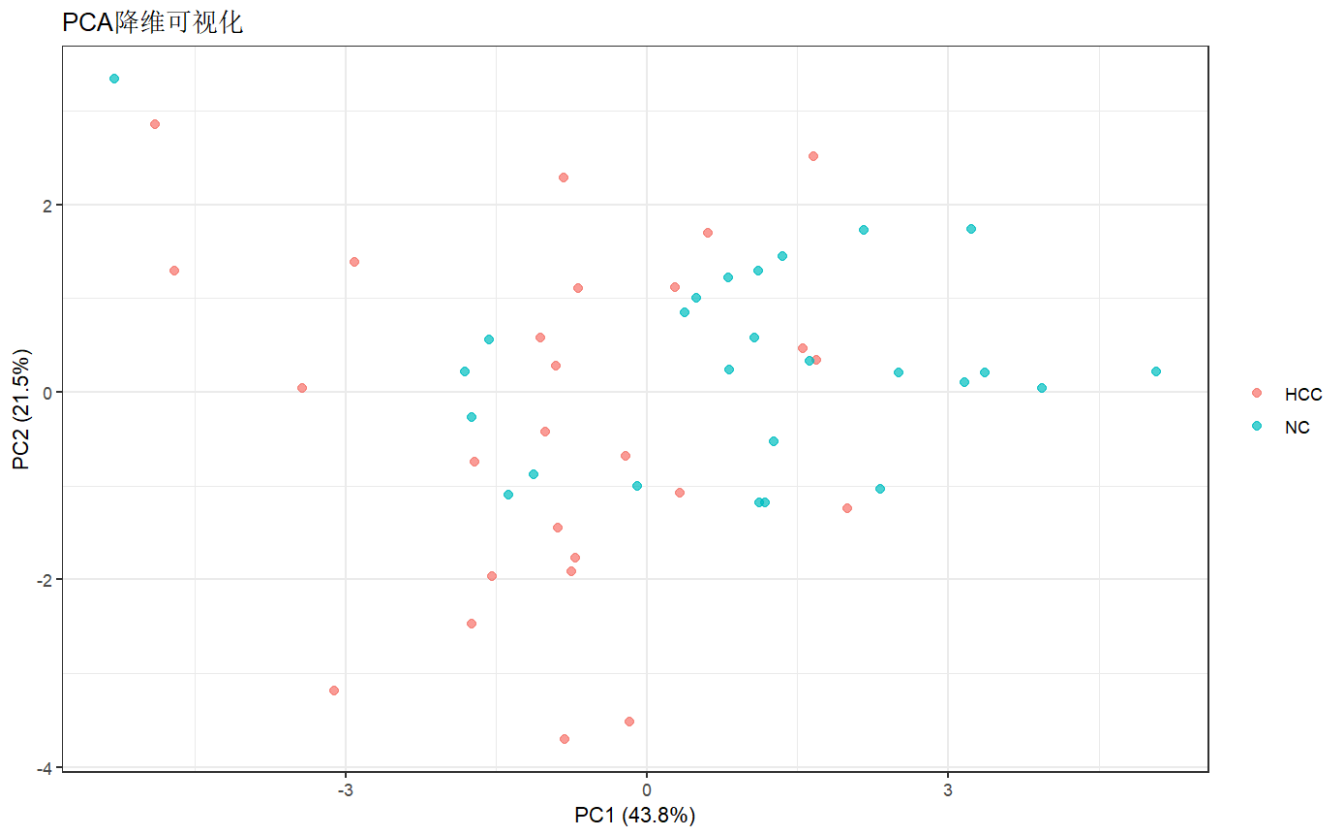
```

# 载入必要的R包
library(Rtsne)
library(ggplot2)
library(randomForest)

# 使用PCA进行可视化
pca.res <- prcomp(x, center = FALSE, scale = FALSE, rank. = 2)
pca_data <- data.frame(
+   PC1 = pca.res$x[, 1], # 第1主成分坐标
+   PC2 = pca.res$x[, 2], # 第2主成分坐标
+   label = y # 样本标签
+ )
ggplot(pca_data, aes(x = PC1, y = PC2, color = label)) +
+   geom_point(size = 2, alpha = 0.7) +
+   theme_bw() +
+   labs(title = "PCA降维可视化",
+        x = paste0("PC1 (", round(summary(pca.res)$importance[2, 1] * 100, 1), "%)"),
+        y = paste0("PC2 (", round(summary(pca.res)$importance[2, 2] * 100, 1), "%)")) +
+   theme(legend.title = element_blank())

```

PCA结果如下图所示:



createDataPartition方法划分数据集:

```
# 固定random seed保证结果可重复
set.seed(421)

# 80% data for training, 20% for performance evaluation
train.indices <- createDataPartition(y,p=0.8,times = 1,list=T)$Resample1
x.train <- x[train.indices,]
x.test <- x[ -train.indices,]
y.train <- y[train.indices]
y.test <- y[ -train.indices]
```

特征选择(递归特征消除(RFE)), 基于随机森林法:

```
# 把twoClassSummary用作模型评估
rfFuncs$summary <- twoClassSummary

# 我们这里使用默认的boot, 即通过bootstrapping(有放回的抽样)进行性能评估
rfectrl <- rfeControl(functions=rfFuncs,
+                       verbose = TRUE,
+                       method="boot",number=25)
rfe.results <- rfe(x.train, factor(y.train),
+                 sizes=2:11,
+                 rfeControl=rfectrl,
+                 metric = "ROC")

# 输出, 保留5个特征
selected.features <- predictors(rfe.results)
```

```
selected.features
[1] "miR.122"      "LINC01226"      "hsa_circ_0080695"
[4] "SNORD3B"      "miR.21"
```

调参:

```
# 使用caret提供的train函数, 以glmnet package实现的带正则化的logistic regression为例通过交叉验证调参
params.grid <- expand.grid(alpha = c(0, 0.5, 1), lambda = c(0, 0.01, 0.1, 1))

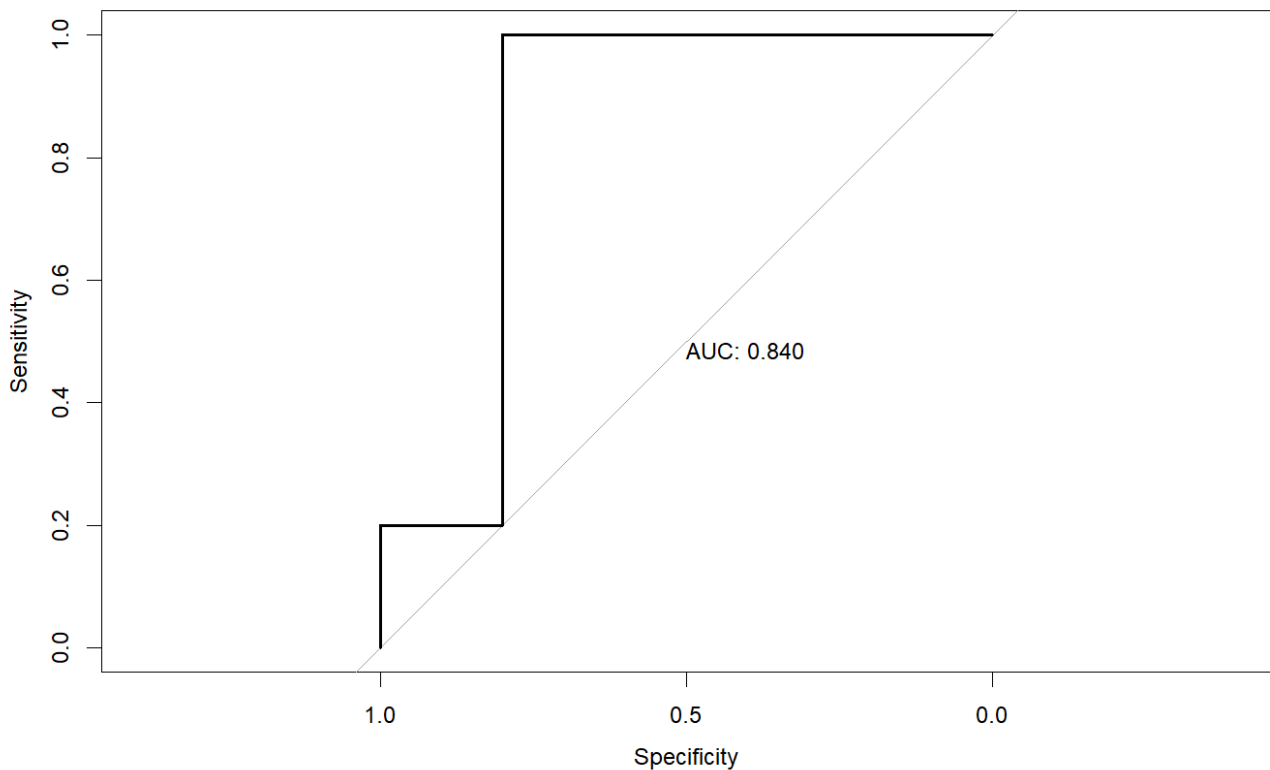
params.grid <- expand.grid(alpha = c(0, 0.5, 1), lambda = c(0, 0.01, 0.1, 1))
tr.ctrl <- trainControl(method = "cv",
+                       number = 10,
+                       summaryFunction = twoClassSummary,
+                       classProbs = TRUE)
cv.fitted <- train(x.train[, predictors(rfe.results)], # 使用RFE筛选的特征
+               y.train, # 训练集标签
+               method = "glmnet", # 使用glmnet包
+               family = "binomial", # 二分类逻辑回归
+               metric = "ROC", # 优化ROC而非准确率
+               tuneGrid = params.grid, # 参数网格
+               preProcess = NULL, # 不额外预处理
+               trControl = tr.ctrl) # 交叉验证控制

# 输出, 超参数推荐值
cv.fitted$bestTune
  alpha lambda
11     1   0.1
```

模型性能评估:

```
# 获取测试集上的预测结果
y.test.pred.prob <- predict(cv.fitted, newdata = x.test, type = "prob")
roc.curve <- roc(y.test, y.test.pred.prob[, 2], levels = c("NC", "HCC"))
plot(roc.curve, print.auc=T)
```

ROC曲线如下图所示：



可见AUC值为0.840，整体曲线在左上角，说明模型评价良好。

2. 随机森林是生物信息经常使用的一个分类器。请大家查阅资料，回答以下两个问题：

- 随机森林中树的数量是不是一个超参数?为什么?

是的。

超参数是模型训练前需要手动设置或通过搜索确定的参数，而不是从训练数据中学习得到的。而随机森林的树的数量需要提前设定。增加 `n_estimators` 通常能提高模型的稳定性和泛化能力，但也会增加训练时间和计算开销。

- 请问什么是随机森林的out-of-bag (OOB) error?它和bootstrapping有什么关系?

OOB误差是使用未被选入Bootstrap样本的数据来评估模型性能。随机森林模型中，训练每棵树时，会使用bootstrap抽样选出部分数据用作训练，剩余数据用于验证，使用这部分数据进行验证得到的误差值就是OOB error。